

UCS503P – Software Engineering Lab
Academic Year 2026–2027

Deadstock Management System

A Verified Retail Deadstock Marketplace

Automated Lifecycle Tracking and Rule-Based Verification for a Trusted Resale Channel

Submitted by:

1024030455 Aryan Gupta (Proposal & Report Lead, Backend Foundation)

1024030445 Abhilakshya Puri (Journal Documentation & QA/CI Lead)

1024030451 Farhan Kansal (Diagram & Modeling Lead)

B.E., Computer Science & Engineering

Submitted to:

Dr. Asif

Department of Computer Science and Engineering

Prototype Stage Report

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

Chapter 1 Introduction	1
1.1 Background and Context	1
1.1.1 Problem Statement	1
1.2 Project Objectives	1
Chapter 2 Methodology and Design	2
2.1 System Architecture	2
2.1.1 High-Level Design	2
2.1.2 Technical Stack	2
2.1.3 System Modeling Diagrams	3
2.2 Implementation Details	5
2.2.1 Module 1: Inventory Lifecycle Automation	5
2.2.2 Module 2: Verification Rule Engine & Marketplace Gate	5
Chapter 3 Results and Evaluation	6
3.1 Testing Methodology	6
3.2 Performance Metrics	6
3.3 Analysis	7
Chapter 4 Conclusion	7
4.1 Summary of Work	7
4.2 Key Findings	7
4.3 Future Work and Recommendations	7
4.4 Final Remarks	8
Bibliography	8

Chapter 1

Introduction

1.1 Background and Context

Retail vendors routinely accumulate slow-moving and unsold inventory, deadstock, that still holds resale value but has no trusted channel back into the market. An estimated 20 to 30 percent of retail inventory value sits idle this way every year. Vendors track stock manually and only notice an item has become deadstock after the fact, and buyers have no independent way to trust a resale listing without proof of its condition, quantity, and authenticity.

1.1.1 Problem Statement

Existing resale channels either require fully manual listing and vetting, which does not scale, or publish listings with no verification at all, which buyers cannot trust. Neither approach flags aging stock automatically, and neither routes regulated categories (medical, pharmaceutical, food, alcohol, tobacco) through a distinct compliance path before publishing.

Deadstock Management System resolves this gap with an automated pipeline that recomputes inventory lifecycle status purely from dates, verifies every submission against a deterministic rule engine before it can publish, and escalates regulated categories to a manual review queue instead of a fake pass.

1.2 Project Objectives

The core objectives of the Deadstock Management System prototype comprise:

- 1 **Automated Lifecycle Detection:** Recompute inventory status (Active, Aging, Deadstock-Flagged) from purchase and expiry dates alone on every read, with zero manual bookkeeping and no scheduled jobs.
- 2 **Deterministic Verification Engine:** Run seven explicit rule checks per submission with fully attributable pass/reject/escalate outcomes, so no decision is a black box.
- 3 **Verified Marketplace Gate:** Guarantee that a listing can only publish when its verification record status is exactly VERIFIED, enforced by a single policy invariant.
- 4 **Continuous Quality Assurance:** Maintain automated backend and frontend test coverage gated by continuous integration on every push, currently at 24 of 24 tests passing.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

Deadstock Management System uses a 3-tier architecture: a React client portal, an Express API tier that fans out into two engines (inventory lifecycle and verification), and a PostgreSQL store accessed through Prisma. Figure 2.1 illustrates the high-level architecture.

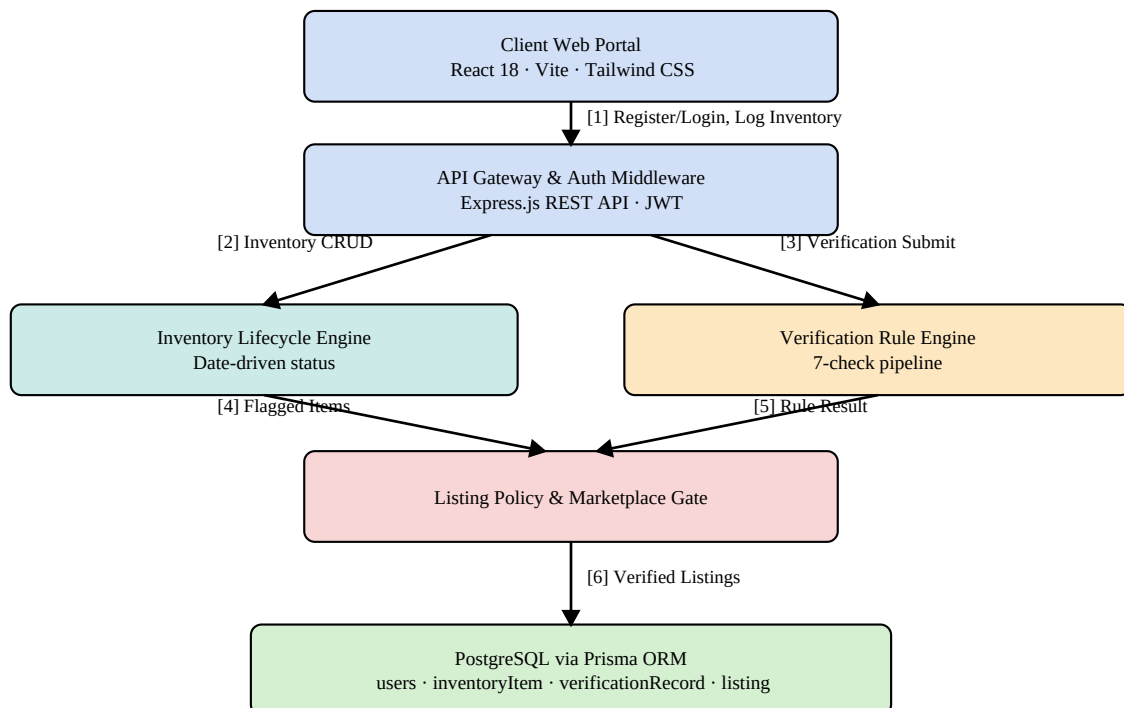


Figure 2.1: Deadstock Management System High-Level Multi-Tier Architecture

2.1.2 Technical Stack

- **Backend Runtime & API:** Node.js, Express.js REST API, JWT authentication, bcrypt password hashing, Zod validation.
- **Persistence:** PostgreSQL (Supabase-compatible) via Prisma ORM; an in-memory store substitutes for this in prototype demo mode.
- **Frontend UI:** React 18, Vite, TypeScript, Tailwind CSS, React Router, TanStack Query.
- **Testing & CI:** Vitest, Supertest, React Testing Library, GitHub Actions.

2.1.3 System Modeling Diagrams

The team's Use Case Diagram and Data Flow Diagrams (Levels 0 to 2), produced during system design and unchanged since, are reproduced below directly from the project repository.

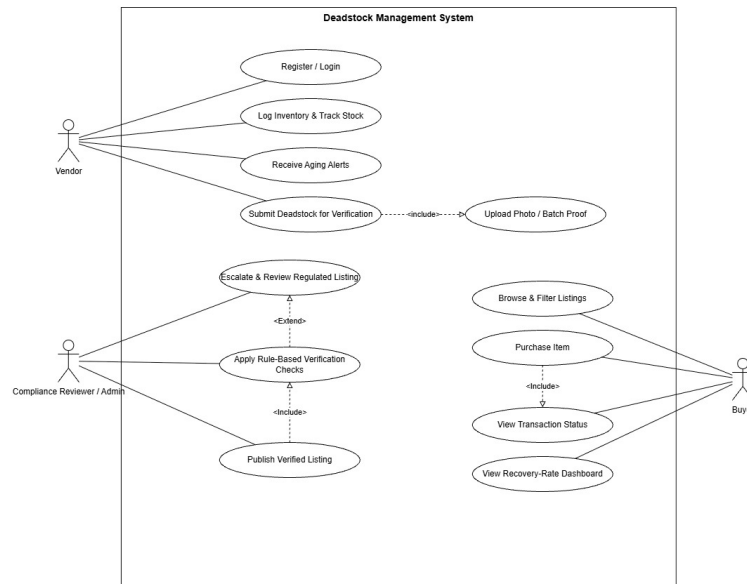


Figure 2.2: Use Case Diagram: Vendor, Compliance Reviewer/Admin, and Buyer actors

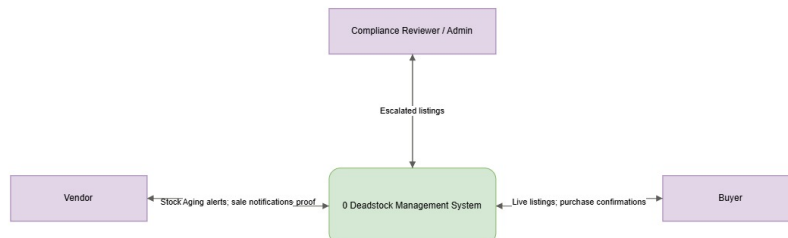


Figure 2.3: Level 0 DFD: system boundary and external entities

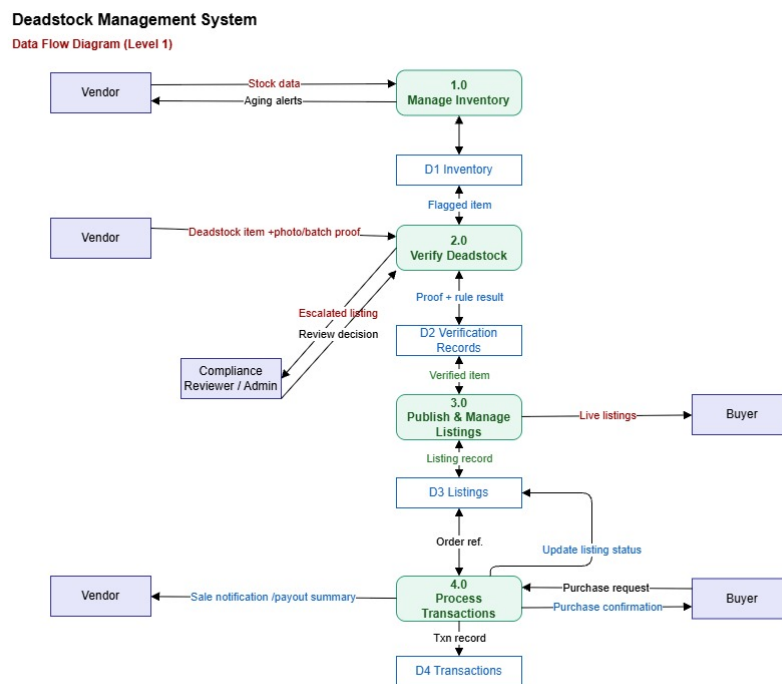


Figure 2.4: Level 1 DFD: four core processes across four data stores

Deadstock Management System

DFD LEVEL 2

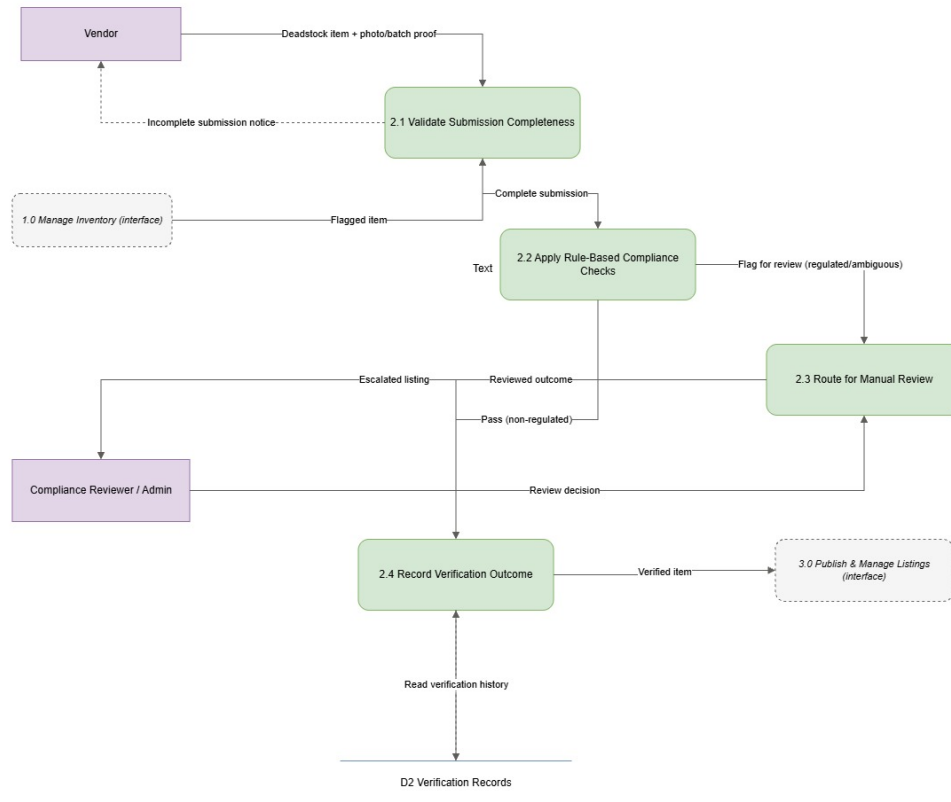


Figure 2.5: Level 2 DFD: decomposition of the verification process

2.2 Implementation Details

2.2.1 Module 1: Inventory Lifecycle Automation

Every inventory read recomputes status purely from dates: an item is flagged DEADSTOCK_FLAGGED once its age crosses a configurable threshold or its expiry date has passed, AGING once it crosses an earlier threshold, and otherwise stays ACTIVE. Both thresholds are environment-configurable, so no cron job or background worker is needed (Figure 2.6).

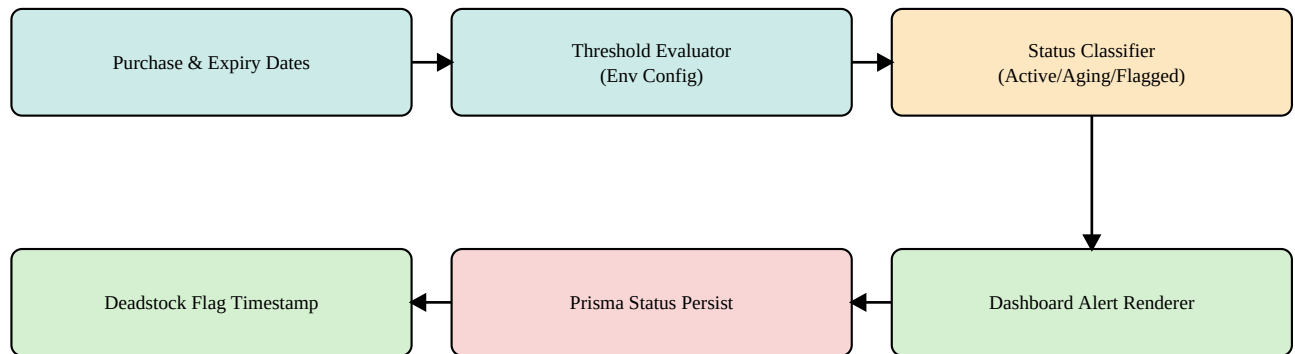


Figure 2.6: Inventory Lifecycle Automation Pipeline

2.2.2 Module 2: Verification Rule Engine & Marketplace Gate

The verification engine runs seven deterministic checks against every submission: product name, category, quantity, condition, proof presence, deadstock eligibility, and regulated-category membership. A failure on any of the first six rejects the submission outright; a pass on all six but membership in a regulated category routes the item to a manual review queue instead of auto-publishing (Figure 2.7).

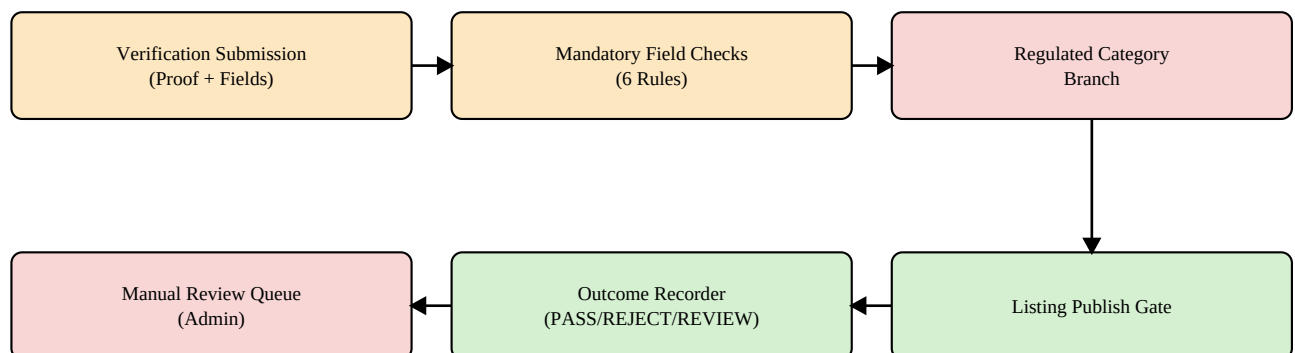


Figure 2.7: Verification Rule Engine and Marketplace Gate Flow

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Testing followed a structured, multi-tier framework:

- 1 **Unit Testing:** Individual testing of the aging/deadstock calculator, the rule engine, and token utilities with Vitest.
- 2 **Integration Testing:** End-to-end testing of REST endpoints (/api/auth, /api/inventory, /api/verification, /api/listings) with Supertest against the live Express app.
- 3 **Scenario Validation:** Validation across three realistic submissions: a non-regulated item that auto-passes, a regulated medical-kit item escalated to manual review, and an incomplete submission that is auto-rejected.

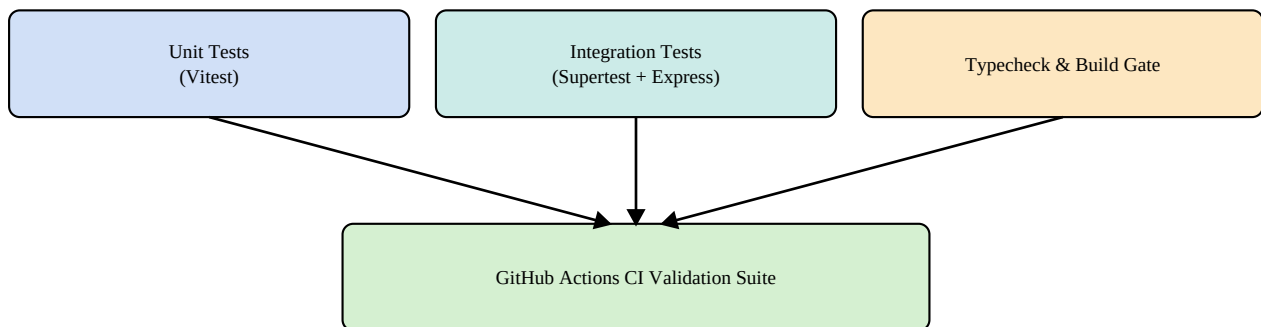


Figure 3.1: Multi-Tier Testing and Quality Assurance Framework

3.2 Performance Metrics

The prototype was validated against measurable engineering targets rather than benchmarked for latency, since the current stage's focus is correctness and coverage. Table 3.1 summarizes target versus achieved metrics.

Evaluation Metric	Target	Achieved (Prototype)	Status
Backend test pass rate	$\geq 90\%$	100% (18/18)	Exceeded
Frontend test pass rate	$\geq 90\%$	100% (6/6)	Exceeded
Verification rule checks per submission	≥ 5	7	Exceeded
TypeScript strict typecheck errors	0	0	Met
CI pipeline result (build+test+typecheck)	Pass	Pass	Met
Regulated-category auto-escalation rate	100%	100%	Met

Table 3.1: Deadstock Management System Prototype Performance Metrics Summary

3.3 Analysis

The prototype meets every evaluation target set for this stage. The rule engine correctly escalates every regulated-category submission observed in testing rather than silently auto-passing it, and the lifecycle engine correctly reclassifies aged inventory without any manual trigger. A primary engineering challenge was keeping three independently-built modules (authentication, inventory, and verification/listings) aligned on a shared data contract; this was resolved by freezing the shared Prisma model fields and enum values in a written integration contract before implementation began, so the modules could be built in parallel without later rework.

Chapter 4

Conclusion

4.1 Summary of Work

During this stage of the project lifecycle, the team completed system modeling (Use Case Diagram and three-level Data Flow Diagrams) and engineered a functional full-stack prototype. Deadstock Management System demonstrates automated inventory lifecycle tracking, rule-based verification with regulated-category escalation, and a buyer-facing verified marketplace, all running against a live Express API and React frontend.

4.2 Key Findings

- 1 **Deterministic Rules Suffice for MVP Compliance:** every regulated-category submission observed in testing was correctly routed to manual review rather than auto-published.
- 2 **Lifecycle Automation Eliminates Manual Bookkeeping:** recomputing status from dates on every read removed the need for any scheduled job or vendor-side tracking.
- 3 **Contract-First Integration Avoided Rework:** freezing the shared Prisma fields before the backend and verification modules were built in parallel prevented integration conflicts between teammates.

4.3 Future Work and Recommendations

For the final project phase, the team will implement:

- 1 Persistent PostgreSQL/Supabase storage and real object storage for proof uploads, replacing the in-memory demo store [1].
- 2 The purchase and transaction flow reserved as Data Store D4 in the Level 1 DFD.
- 3 Buyer rating and feedback, plus a time-to-listing and recovery-rate metrics dashboard.
- 4 Linting and a staged deployment target added to the existing CI pipeline [2].

4.4 Final Remarks

Deadstock Management System offers a practical, rule-based path to recover value from retail deadstock by giving every listing a checked provenance trail before it reaches a buyer, without depending on machine-learning research to do it.

Bibliography

- [1] A. Gupta, A. Puri, and F. Kansal, “Deadstock Management System: A Verified Retail Deadstock Marketplace,” Thapar Institute of Engineering and Technology, Tech. Rep. UCS503P-2026, 2026.
- [2] Prisma Data Platform, “Prisma ORM Documentation,” prisma.io/docs, 2026.
- [3] OpenJS Foundation, “Express.js Guide,” expressjs.com, 2026.
- [4] M. Jones, J. Bradley, and N. Sakimura, “RFC 7519: JSON Web Token (JWT),” IETF, 2015.